

1 CUDA: варп шафлы и общая память

В предыдущей лекции рассматривалась работа с глобальной памятью графической карты — самой вместительной, но одновременно и самой медленной. В этой лекции будет продемонстрирована работа с общей памятью для всех потоков внутри SM. Также будет показано, как поток может «заглянуть» в регистровую память своих «соседей» по варпу.

В качестве примера рассмотрим программу для вычисления значения определенного интеграла (листинг 1).

Листинг 1: Метод трапеций: первая версия

```
#include <stdio.h>
#include <cuda.h>

// h*(0.5(f(a)+f(b))+f(x1)+f(x2)+...f(xn))

__device__ __host__ float f(float x)
{
    return x * x;
}

__global__ void Trap(float a, float b, float h, int n, float *trap_p)
{
    int my_i = blockDim.x * blockIdx.x + threadIdx.x;
    if (0 < my_i && my_i < n)
    {
        float my_x = a + my_i * h;
        float my_trap = f(my_x);
        atomicAdd(trap_p, my_trap);
    }
}

int main(int argc, char *argv[])
{
    float a = strtol(argv[1], NULL, 10);
    float b = strtol(argv[2], NULL, 10);
    int n = strtol(argv[3], NULL, 10);
    int blk_ct = strtol(argv[4], NULL, 10);
    int th_per_blk = strtol(argv[5], NULL, 10);

    float *trap_p;
    cudaMallocManaged(&trap_p, sizeof(float));
    *trap_p = 0.5 * (f(a) + f(b));
    float h = (b - a) / n;

    Trap<<<blk_ct, th_per_blk>>>>(a, b, h, n, trap_p);
```

```

    cudaDeviceSynchronize();

    *trap_p = h>(*trap_p);

    printf("%f\n", *trap_p);

    return 0;
}

```

1.1 Варп шафл

Программа в листинге 1 обладает существенным недостатком: в ней потоки одновременно считают каждый свое слагаемое, а формирование суммы происходит уже последовательно.

Решим эту проблемы следующим образом. Набор из 32 потоков с идущими подряд индексами (`threadIdx.x`) называется варпом. Все потоки внутри варпа работают как единая SIMD-система.

Предположим, что каждый блок состоит только из 32 потоков (одного варпа). Тогда реализуем древовидную схему суммирования, аналогичную функции `Reduce` в MPI. Для этого будем использовать функцию `__shfl_down_sync`, которая позволяет потоку «заглянуть» в регистровую память другого потока в том же самом варпе.

```

__device__ float __shfl_down_sync(
    unsigned mask,
    float var,
    unsigned diff
);

```

- **mask** — 32-битное число, каждый бит которого означает, участвует ли соответствующий поток в коммуникации.
- **var** — имя переменной, значение которой мы хотим узнать.
- **diff** — положительное число, разница между номером потока в варпе, у которого мы смотрим значение **var**, и номером текущего потока в варпе.
- Возвращаемое значение — это значение переменной **var** на потоке $l + \text{diff}$, где l — номер текущего потока в варпе, т.е. $l = \text{threadIdx.x} \% \text{warpSize}$.

Важно помнить про следующие особенности работы рассматриваемой функции:

1. Если поток с рангом l вызывает функцию `__shfl_down_sync`, а поток $l + \text{diff}$ не вызывает, то поведение программы не определено.

2. Если поток с рангом l вызывает функцию `__shfl_down_sync`, но $l + \text{diff} \geq \text{warpSize}$, тогда функция вернет значение переменной `var` с потока l (текущего потока).
3. Если поток с рангом l вызывает функцию `__shfl_down_sync`, и $l + \text{diff} < \text{warpSize}$, однако количество потоков в данном конкретном варпе меньше $\text{warpSize} = 32$. такая ситуация возникает, когда количество потоков в блоке не кратно `warpSize`. В этом случае поведение не определено.

Таким образом, для безопасной работы варп шафла, нужно выполнять следующие правила:

1. Все потоки, участвующие в коммуникации, должны вызывать `__shfl_down_sync`.
2. Количество потоков в блоке должно быть кратным `warpSize`.

Реализация метода трапеций с использованием варп шафла показана в листинге 4.

Листинг 2: Метод трапеций (варп шафл)

```
__device__ float Warp_sum(float var)
{
    int mask = 0xffffffff;
    for (int diff = warpSize / 2; diff > 0; diff /= 2)
        var += __shfl_down_sync(mask, var, diff);
    return var;
}

__global__ void TrapWarp(float a, float b, float h, int n, float *trap_p)
{
    int my_i = blockDim.x * blockIdx.x + threadIdx.x;
    float my_trap = 0.0f;
    if (0 < my_i && my_i < n)
    {
        float my_x = a + my_i * h;
        my_trap = f(my_x);
    }
    float result = Warp_sum(my_trap);
    if (threadIdx.x == 0)
        atomicAdd(trap_p, result);
}
```

Функционирование варп шафла показано на рисунке 1.

1.2 Общая память

Переменная, размещаемая в общей памяти (общей для всех процессоров внутри SM, но отдельная для каждой SM), обозначается при объявлении идентификатором `__shared__`.

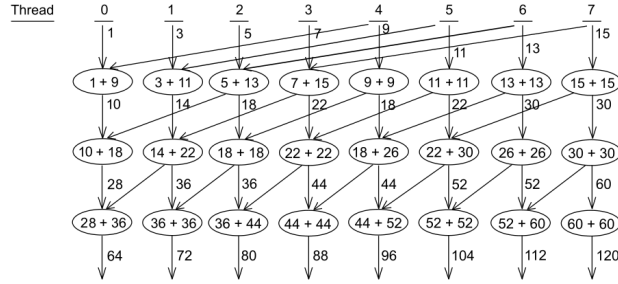


Рис. 1: Работа варп шафла

Воспользуемся массивом в общей памяти для того, чтобы осуществлять суммирование внутри варпа, см. листинг 3. Механизм суммирования немного отличается, см. рис. 2.

Листинг 3: Метод трапеций (общая память)

```
__device__ float Shared_mem_sum(float shared_vals[])
{
    int my_lane = threadIdx.x % warpSize;
    for (int diff = warpSize / 2; diff > 0; diff /= 2)
    {
        int source = (my_lane + diff) % warpSize;
        shared_vals[my_lane] += shared_vals[source];
    }
    return shared_vals[my_lane];
}

__global__ void TrapShared(float a, float b,
float h, int n, float *trap_p)
{
    __shared__ float shared_vals[32];
    int my_i = blockDim.x * blockIdx.x + threadIdx.x;
    int my_lane = threadIdx.x % warpSize;
    shared_vals[my_lane] = 0.0f;
    if (0 < my_i && my_i < n)
    {
        float my_x = a + my_i * h;
        shared_vals[my_lane] = f(my_x);
    }
    float result = Shared_mem_sum(shared_vals);
    if (threadIdx.x == 0)
        atomicAdd(trap_p, result);
}
```

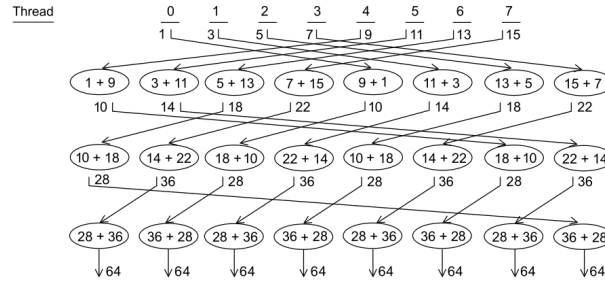


Рис. 2: Суммирование с использованием общей памяти.

2 CUDA: синхронизация потоков и банки памяти

2.1 Синхронизация потоков внутри блока и банки памяти

Программы в листингах 4 и 3 страдают одним и тем же недостатком: в блоке может быть только один варп (32 потока), хотя на самом деле их может быть до 32 (1024 потока). Программа из листинга 4 не будет работать при большем количестве потоков, потому что варп шафл позволяет суммировать значения только внутри варпа. Если варпов несколько, то продемонстрированный на рисунке 1 просто не будет работать, ибо коммуникация между варпами невозможна. Если же мы посмотрим на программу из листинга 3 и изменим размер общего массива с 32 до 1024 (или другого количества потоков), то мы обнаружим, что результат недетерминирован, т.к. потоки из разных варпов не синхронизированы.

Следовательно, для запуска программы при количестве потоков в блоке, большем, чем 32, нам необходимо по каждому варпу посчитать каждую сумму, а затем, в нулевом потоке каждого варпа, прибавить эту локальную сумму к общей сумме по блоку. Далее нулевой поток в блоке прибавляет локальную сумму блока к общей сумме. Беда вся в том, что потоки из разных варпов работают каждый в своем темпе и никак не синхронизируются. Поэтому, когда нулевой поток в блоке будет прибавлять свою сумму к глобальной сумме, может оказаться, что прибавляемая локальная сумма еще не сформирована (не все варпы записали в нее свои результаты).

Синхронизация потоков внутри одного блока производится посредством вызова в каждом потоке функции `__syncthreads()`.

Остается решить вопрос, где хранить результаты по варпам, прежде чем просуммировать их друг с другом. Есть три варианта решения вопроса:

1. Создать новый массив из 32 элементов. Нулевой поток в i -м варпе пишет свой результат в i -й элемент массива. Далее барьер (ждем, пока все варпы завершат свою работу). Находим сумму элементов нового

массива.

2. Создать единый массив на 1024 элемента. Поток с рангом i пишет свой результат в i -й элемент массива. Нулевой поток в i -м варпе считает сумму по варпу и записывает ее в i -й элемент массива. Но перед этим нужно поставить барьер, ведь нам нужно ждать, пока нулевой варп посчитает свои результаты и запишет сумму в нулевой элемент массива. Иначе другие варпы могут записать свои результаты в первые 32 элемента массива прежде, чем нулевой варп завершит свою работу (гонка). Далее, еще один барьер, в котором мы ждем, пока все варпы заполнят первые 32 элемента массива. Далее находим сумму первых 32 элементов массива.
3. Все то же, что в предыдущем пункте, однако хранить результаты по варпам будем не в первых 32 элементах общего массива, а в каждом 32 его элементе.

Первый вариант требует дополнительную общую память для 32 флюатов. Всего, таким образом, нужно 1056 4-байтовых чисел. Этот вариант нас устраивает, т.к. общая память исчисляется десятками килобайт.

Второй вариант позволяет немного сэкономить память, но требует две синхронизации по блокам вместо одной.

Третий вариант решает проблему второго, ибо ему достаточно одной синхронизации. Однако здесь необходимо помнить про банки памяти. Общая память внутри SM организована в банки. Первый банк содержит элементы массива с индексами 0, 31, 63 и далее. Первый банк элементы 1, 32, 64, ..., и.т.д. Доступ к разным банкам памяти может быть одновременным, но доступ к одному банку памяти только последовательный. Поэтому третий вариант, как и второй, будет несколько медленнее, чем первый.

В листинге 4 приведен фрагмент программы, реализующей первую из рассмотренных выше идей.

Листинг 4: Метод трапеций (несколько варпов в блоке)

```
__device__ float Warp_sum(float var)
{
    int mask = 0xffffffff;
    for (int diff = warpSize / 2; diff > 0; diff /= 2)
        var += __shfl_down_sync(mask, var, diff);
    return var;
}

__device__ float Shared_mem_sum(float shared_vals[])
{
    int my_lane = threadIdx.x % warpSize;
    for (int diff = warpSize / 2; diff > 0; diff /= 2)
    {
        int source = (my_lane + diff) % warpSize;
```

```

        shared_vals[my_lane] += shared_vals[source];
    }
    return shared_vals[my_lane];
}

__global__ void TrapWarp(float a, float b,
    float h, int n, float *trap_p)
{
    __shared__ float warp_sum_arr[WARPSZ];
    int my_warp = threadIdx.x / warpSize;
    int my_lane = threadIdx.x % warpSize;
    int my_i = blockDim.x * blockIdx.x + threadIdx.x;
    float my_trap = 0.0f;
    if (0 < my_i && my_i < n)
    {
        float my_x = a + my_i * h;
        my_trap = f(my_x);
    }
    float temp = Warp_sum(my_trap);
    if (my_lane == 0)
        warp_sum_arr[my_warp] = temp;
    __syncthreads();
    float blk_result = 0.0f;
    if (my_warp == 0)
    {
        if (threadIdx.x > blockDim.x / warpSize)
            warp_sum_arr[threadIdx.x] = 0.0;
        blk_result = Shared_mem_sum(warp_sum_arr);
    }

    if (threadIdx.x == 0)
        atomicAdd(trap_p, blk_result);
}

```

2.2 Кооперативные группы

Функция `__syncthreads()` синхронизирует потоки только внутри блока. Но что делать, если хочется синхронизировать все потоки, во всех блоках? Есть два решения проблемы:

1. Реализовать синхронизацию искусственно, разделить единый кернел на несколько. Тогда можно будет вызвать `tt.cudaDeviceSynchronize` с девайса.
2. Использовать кооперативные группы.

Для работы с кооперативными группами нужно подключить заголовочный файл и не забыть про пространство имен:

```
#include <cooperative_groups.h>
using namespace cooperative_groups;
```

Основной тип данных это `thread_group`, у которого есть функции

1. `size()` — количество потоков в группе;
2. `rank()` — номер текущего потока в группе;
3. `is_valid()` — `true`, если потоком можно пользоваться;
4. `sync()` — синхронизация всех потоков внутри группы.

Как создать кооперативную группу?

1. Группа, состоящая из всех потоков внутри блока:
`thread_block block = this_thread_block();`
2. Группа, состоящая из всех потоков внутри грида:
`thread_group grid = this_grid();`
3. Можно разбить существующий блок на подблоки с помощью функции `cg::tiled_partition()`. Так, разбиение блока на варпы может выглядеть следующим образом:

```
thread_group tile32 =
    cg::tiled_partition(this_thread_block(), 32);
```

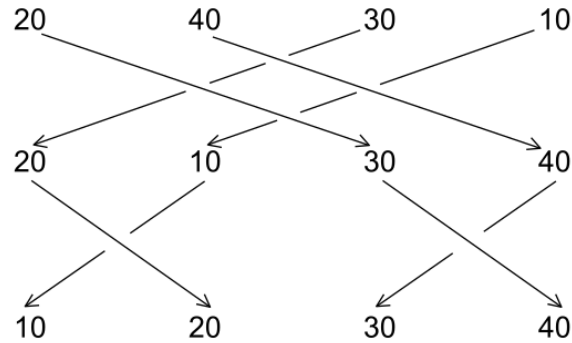


Рис. 3: Битоническая сортировка для 4 элементного массива

3 CUDA: синхронизация потоков (семинар)

Зададимся целью реализовать сортировку массива, используя метод бабочки, или битонной сортировки. Предполагается, что длина массива является степенью двойки.

Для массива длиной 2 алгоритм очевиден (сравниваем и меняем местами элементы).

Для массива длины 4 работа алгоритма продемонстрирована на рис.

Для массива длины 8 работа алгоритма показана на рис.

Последовательный код показан в листинге 5.

Листинг 5: Последовательная программа битонической сортировки

```

unsigned Which_bit(unsigned stage)
{
    unsigned bit = 0;
    while (stage > 0)
    {
        bit++;
        stage = stage >> 1;
    }
    return bit - 1;
}

unsigned insert_zero(unsigned th, unsigned which_bit)
{
    unsigned left_ones = (~0) << which_bit;
    unsigned right_ones = ~left_ones;
    unsigned left_bits = left_ones & th;
    unsigned right_bits = right_ones & th;
    return (left_bits << 1) | right_bits;
}

```

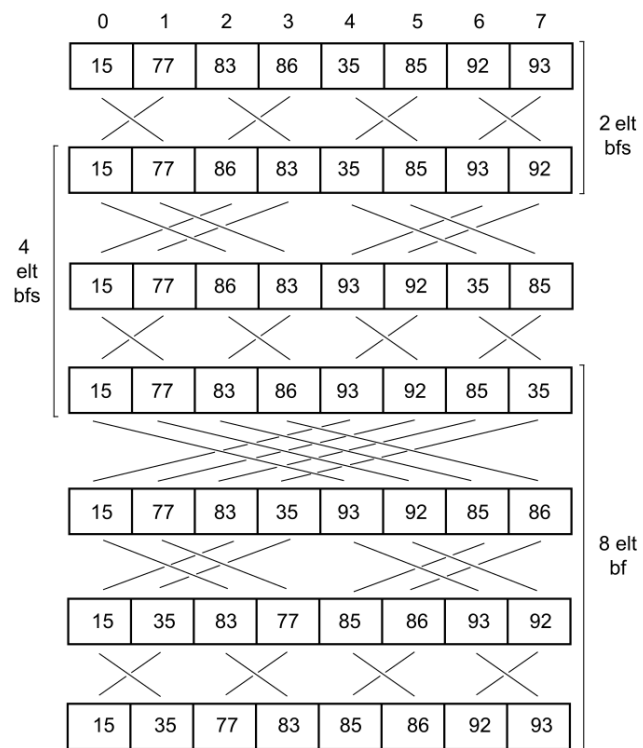


Рис. 4: Битоническая сортировка для 8 элементного массива

```

}

void compare_swap(int arr[], unsigned my_elt1,
                 unsigned my_elt2, unsigned flag)
{
    if (flag == 0)
    {
        if (arr[my_elt1] > arr[my_elt2])
        {
            int tmp = arr[my_elt1];
            arr[my_elt1] = arr[my_elt2];
            arr[my_elt2] = tmp;
        }
    }
    else
    {
        if (arr[my_elt1] < arr[my_elt2])
        {
            int tmp = arr[my_elt1];
            arr[my_elt1] = arr[my_elt2];
            arr[my_elt2] = tmp;
        }
    }
}

void bitone(int *arr, int n)
{
    for (unsigned bf_sz = 2; bf_sz <= n; bf_sz *= 2)
    {
        for (unsigned stage = bf_sz / 2; stage > 0; stage /= 2)
        {
            for (int th=0; th<n/2; th++){
                unsigned which_bit = Which_bit(stage);
                unsigned my_elt1 = insert_zero(th, which_bit);
                unsigned my_elt2 = my_elt1 ^ stage;
                compare_swap(arr, my_elt1, my_elt2, my_elt1 & bf_sz);
            }
        }
    }
}

```

Распараллеливание кода с листинга 5 тривиально, см. листинг 6.

Листинг 6: Параллельная битоническая сортировка (только один блок!)

```

__global__ void bitone(int *arr, int n)
{

```

```

unsigned th = blockDim.x * blockIdx.x + threadIdx.x;
for (unsigned bf_sz = 2; bf_sz <= n; bf_sz *= 2)
{
    for (unsigned stage = bf_sz / 2; stage > 0; stage /= 2)
    {
        unsigned which_bit = Which_bit(stage);
        unsigned my_elt1 = insert_zero(th, which_bit);
        unsigned my_elt2 = my_elt1 ^ stage;
        compare_swap(arr, my_elt1, my_elt2, my_elt1 & bf_sz);
        __syncthreads();
    }
}

```

Беда вся в том, что этот код работает только для одного блока, ведь `__syncthreads()` работает только внутри блока. Поэтому этот код может работать только для массива до 2048 элементов.

Продemonстрируем два способа решить проблему:

1. Код в теле цикла заменить на вызов кернела. Обратите внимание, что в данном случае даже не нужно писать `cudaDeviceSynchronize`, т.е. ни один поток не запустит кернел, пока все не закончили его выполнять на предыдущей итерации цикла. См. листинг 7.
2. Кооперативные группы. См. листинг 8.

Листинг 7: Битоническая сортировка, несколько блоков

```

__global__ void bitone_step(int *arr, int stage, int bf_sz)
{
    unsigned th = blockDim.x * blockIdx.x + threadIdx.x;
    unsigned int which_bit = Which_bit(stage);
    unsigned int my_elt_1 = insert_zero(th, which_bit);
    unsigned int my_elt_2 = my_elt_1 ^ stage;
    Compare_swap(arr, my_elt_1, my_elt_2, my_elt_1 & bf_sz);
}

__host__ void bitone(int *arr, int n,
int block_cnt, int th_per_blk)
{
    for (unsigned bf_sz = 2; bf_sz <= n; bf_sz *= 2)
    for (unsigned stage = bf_sz / 2; stage > 0; stage /= 2)
    {
        bitone_step<<<block_cnt, th_per_blk>>>(arr, stage, bf_sz);
        //cudaDeviceSynchronize();
    }
}

```

Листинг 8: Битоническая сортировка, кооперативные группы

```
__global__ void bitone(void **args)
{
    grid_group grid = this_grid();
    long *arr = (long *)args;
    long n = 2 * grid.num_threads();
    unsigned th = grid.thread_rank();
    for (unsigned bf_sz = 2; bf_sz <= n; bf_sz *= 2)
        for (unsigned stage = bf_sz / 2; stage > 0; stage /= 2)
        {
            unsigned int which_bit = Which_bit(stage);
            unsigned int my_elt_1 = insert_zero(th, which_bit);
            unsigned int my_elt_2 = my_elt_1 ^ stage;
            Compare_swap(arr, my_elt_1, my_elt_2, my_elt_1 & bf_sz);
            grid.sync();
        }
}
```